



**Arab Academy for Science & Technology &
Maritime Transport**

College of Artificial Intelligence

**Data Science Department
DS311 Advanced data base**

Laptop Price Prediction & Analytics System

Supervised By

**Prof .Osama Badawy
Eng. Mazen aziz**

By

**Karim gaber
221003403**

**Ali Al-Shafi
231000323**

AGENDA

- **Introduction & System Overview**
- **Dataset Description**
- **Schema Design & ERD**
- **Frontend Pages & User Flow**
- **Data Preprocessing**
- **Machine Learning Model**
- **Model Deployment**
- **CRUD Operations**
- **Indexing & Aggregation Pipelines**
- **Transactions, Replication**
- **Conclusion**

Introduction & System Overview

- This Project Is An Advanced Database & Machine Learning System
- The System Analyzes E-commerce Laptop Listings Data
- It Combines:
 - **Mongodb (Nosql Database)**
 - **Fastapi (Backend)**
 - **Machine Learning (Price Prediction)**
 - **React (Frontend Dashboard)**

Introduction & System Overview

- Store And Manage Large-scale Dataset
- Perform Analytics And Aggregations
- Predict Laptop Prices Using ML
- Provide A User-friendly Dashboard

Dataset Description

- **DATASET SOURCE**
- THE DATASET WAS COLLECTED USING **WEB SCRAPING TECHNIQUES**
- DATA WAS SCRAPED FROM **E-COMMERCE PLATFORMS** SELLING LAPTOPS
- EACH RECORD REPRESENTS **ONE LAPTOP LISTING**
- **DATASET SIZE & STRUCTURE**
- TOTAL RECORDS: **5,000 LAPTOP LISTINGS**
- DATA TYPE: **STRUCTURED / SEMI-STRUCTURED**
- STORED AS **MONGODB DOCUMENTS (JSON FORMAT)**

Dataset Description

- **Main Data Attributes**
- **Product Information** : Title, Price, Condition
- **Hardware Specifications** : RAM_GB, Storage_gb, Processor, Screen_size_inch
- **Seller Information** : Seller_name, Seller_score, Seller_reviews
- **Logistics & Location** : Shipping_cost, Country
- **Metadata** : Created_at, Updated_at
- Represents **Real-world E-commerce Data**

Schema Design

- **Users Collection**
- **Purpose:** Store System Users & Authentication Data
- User ID
- Name
- Email
- Password (Hashed)
- Created At

Schema Design

- **DATASET COLLECTION**
- **PURPOSE:** STORE SCRAPED LAPTOP LISTINGS (TRAINING & ANALYTICS DATA)
- TITLE , PRICE
- RAM (GB) ,STORAGE (GB)
- PROCESSOR ,SCREEN SIZE (INCH)
- CONDITION , SHIPPING COST
- SELLER SCORE ,SELLER REVIEWS
- COUNTRY ,LISTING TYPE
- CREATED AT

Schema Design

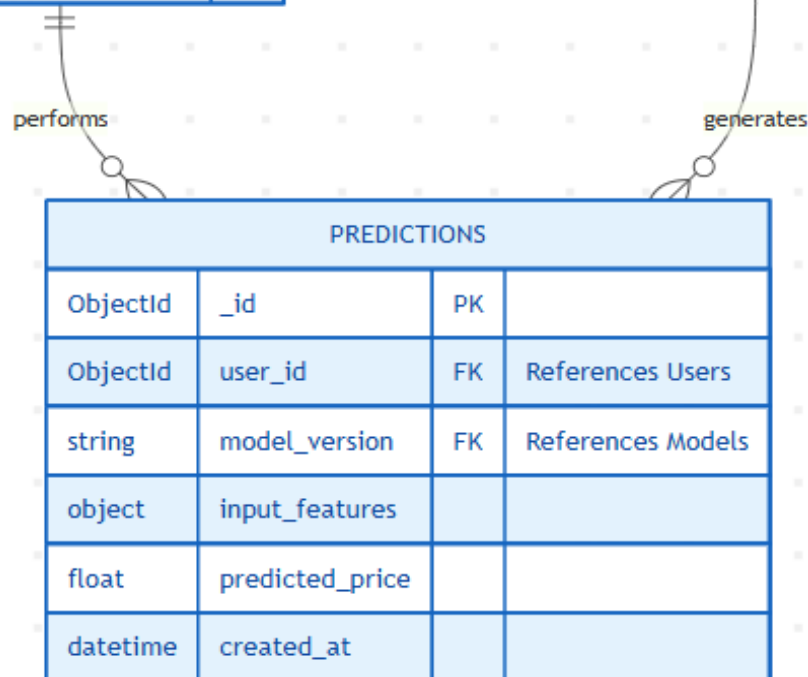
- **Predictions Collection**
- **Purpose:** Store ML Predictions Per User
- Prediction ID
- User ID (*Reference To Users*)
- Input Features
- Predicted Price
- Model Version
- Created At

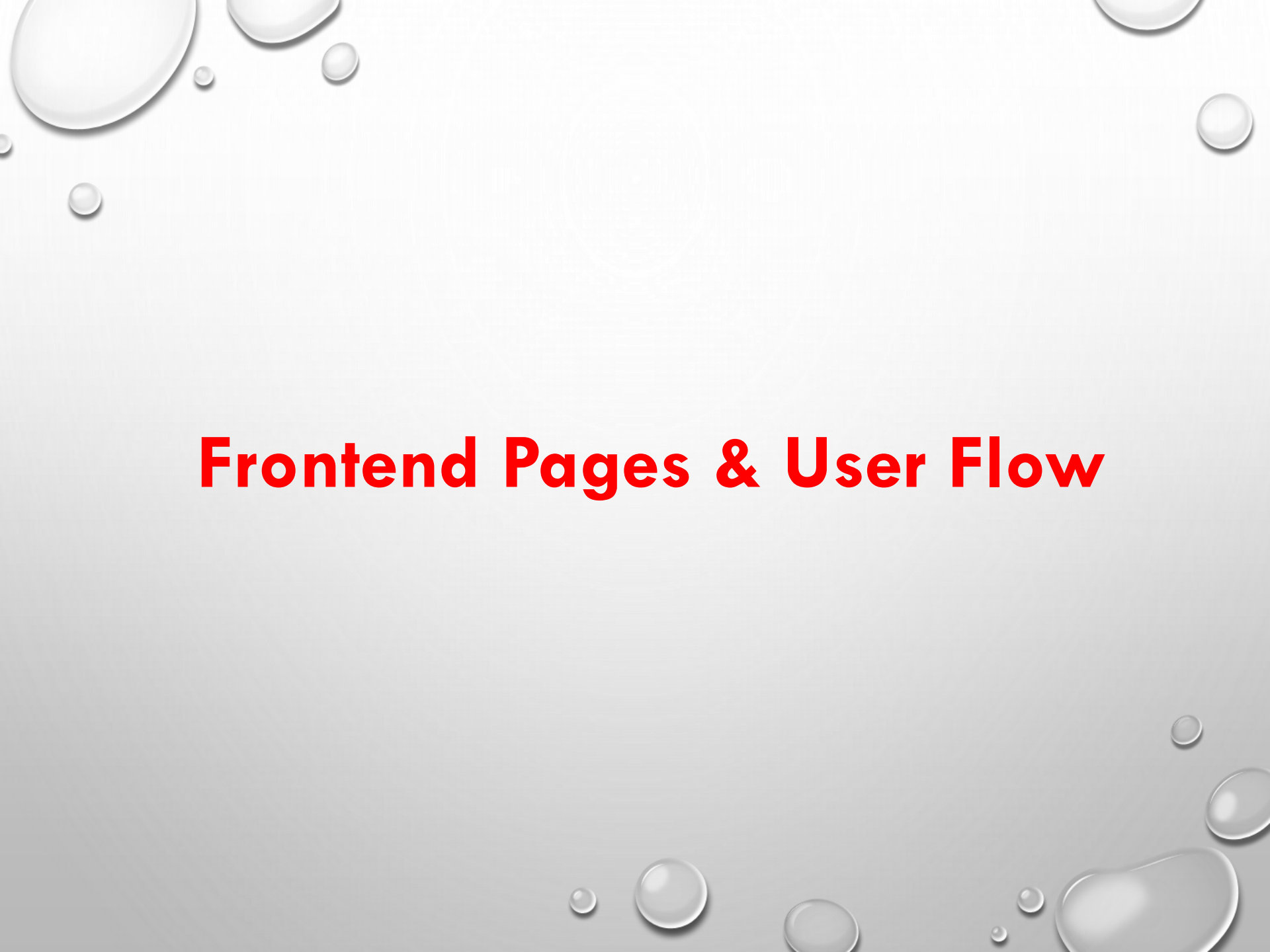
ERD

USERS		
ObjectId	_id	PK
string	name	
string	email	
string	password_hash	
datetime	created_at	

MODELS		
string	version	PK
float	accuracy	
datetime	trained_at	
object	hyperparameters	

DATASET		
ObjectId	_id	PK
string	title	
float	price	
int	ram_gb	
string	processor	
string	condition	



The background is a light gray gradient. It features several realistic water droplets of various sizes, some with highlights and shadows, scattered around the edges. In the center, there is a faint, circular logo or watermark that is not clearly legible.

Frontend Pages & User Flow

Advanced DB Project
MongoDB Atlas • FastAPI • ML

Create an account

Sign up to access analytics, predictions, and your profile.

Name
Kareem

Email
omarbear123@gmail.com

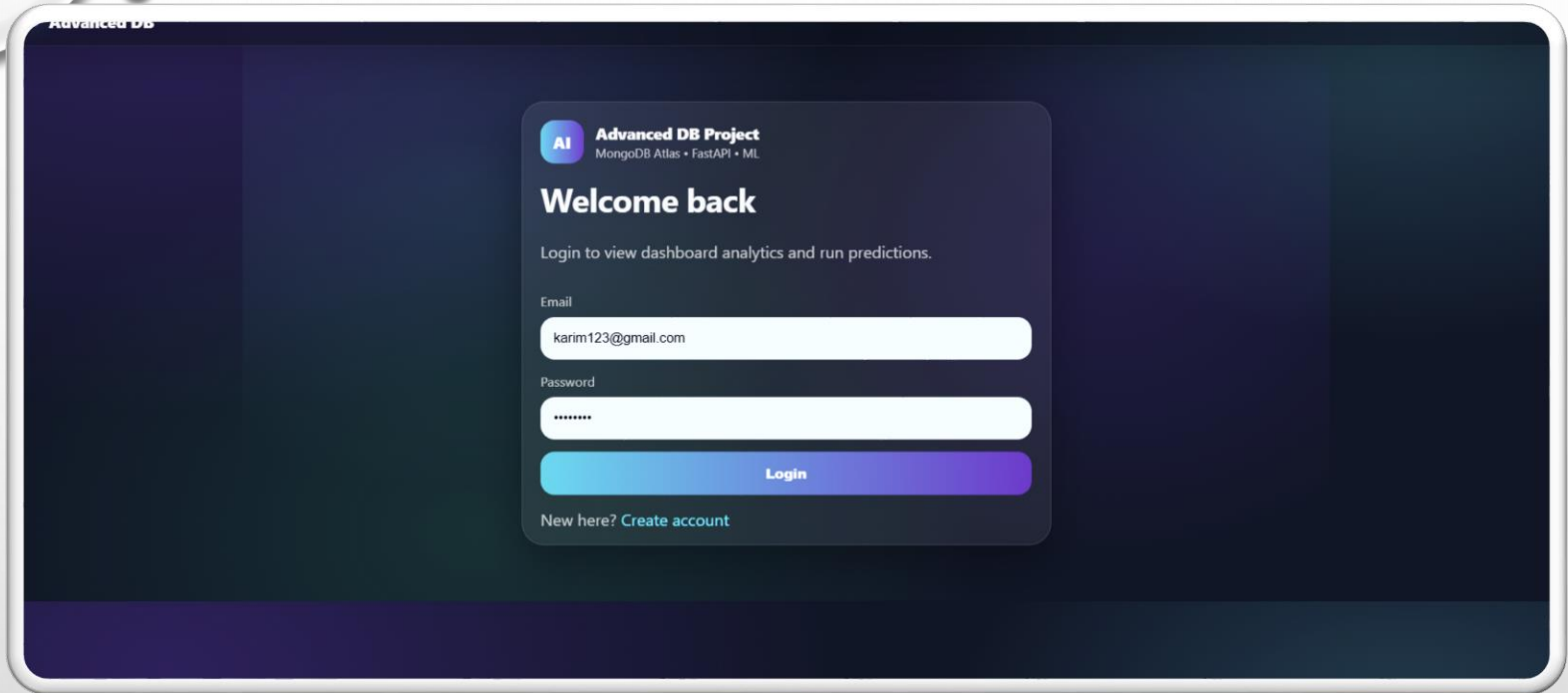
Password

[Sign Up](#)

Already have an account? [Login](#)

SIGNUP PAGE

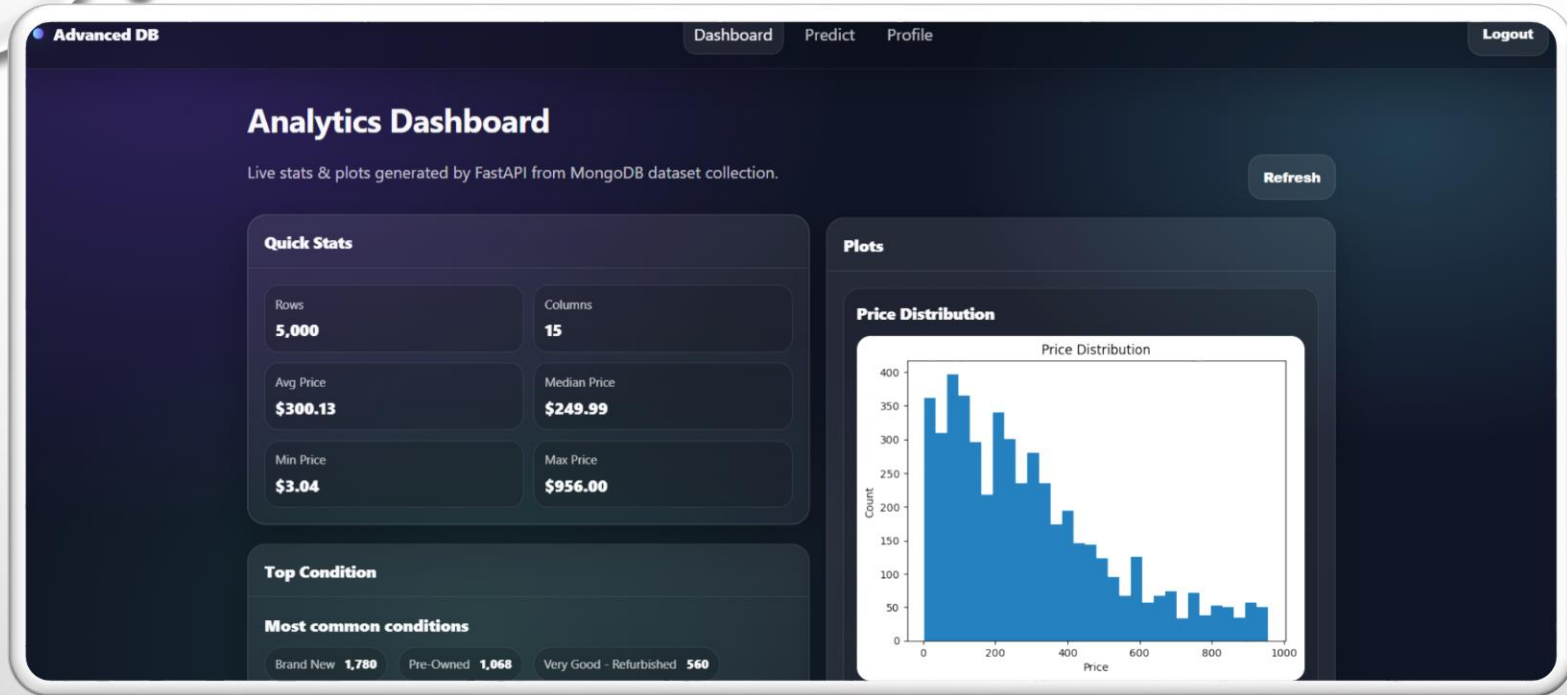
- Allows New Users To Create An Account Using Name, Email, And Password
- Stores User Data Securely In Mongodb And Redirects After Success



LOGIN PAGE

Authenticates Users Using Email And Password

Generates A JWT Token To Access Protected Pages



DASHBOARD PAGE

Displays Dataset Analytics And Statistics From MongoDB

Shows Visual Plots Generated By Backend Aggregation

Advanced DB

Dashboard Predict Profile

omarbear123@gmail.com Logout

AI Insights — Predict Price

RAM_GB	Storage_GB
16	512
Screen_Size_Inch	Shipping_Cost
15.6	0
Seller_Score	Seller_Reviews
98	1200
Is_SSD (0/1)	Cpu_Generation
1	11
Processor	Condition
i7	New
Country	Listing_Type_Cat
US	BuyItNow

Predict

Result

406.33

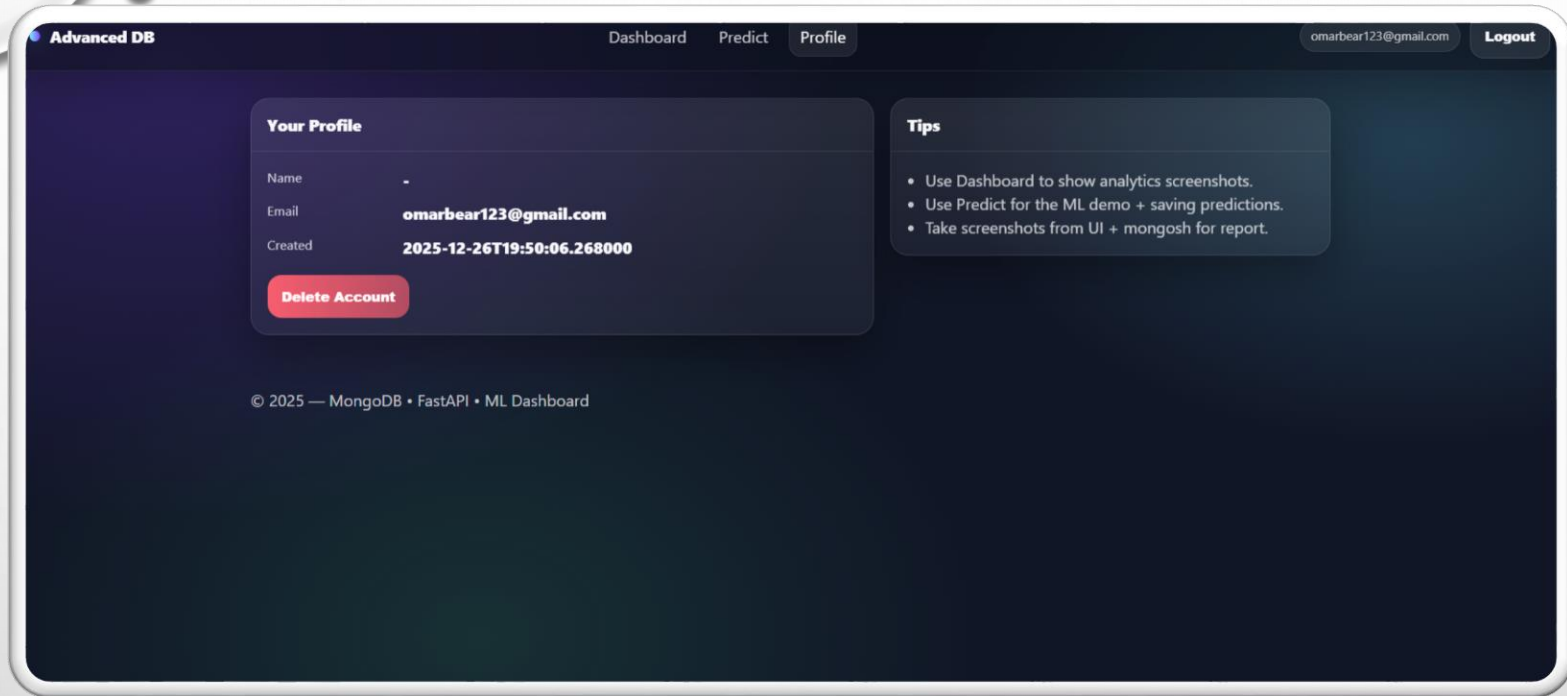
Model: rf_v1

Prediction is saved to **predictions** collection in MongoDB.

ML PREDICTION PAGE

ACCEPTS LAPTOP FEATURES TO PREDICT PRICE USING ML
MODEL

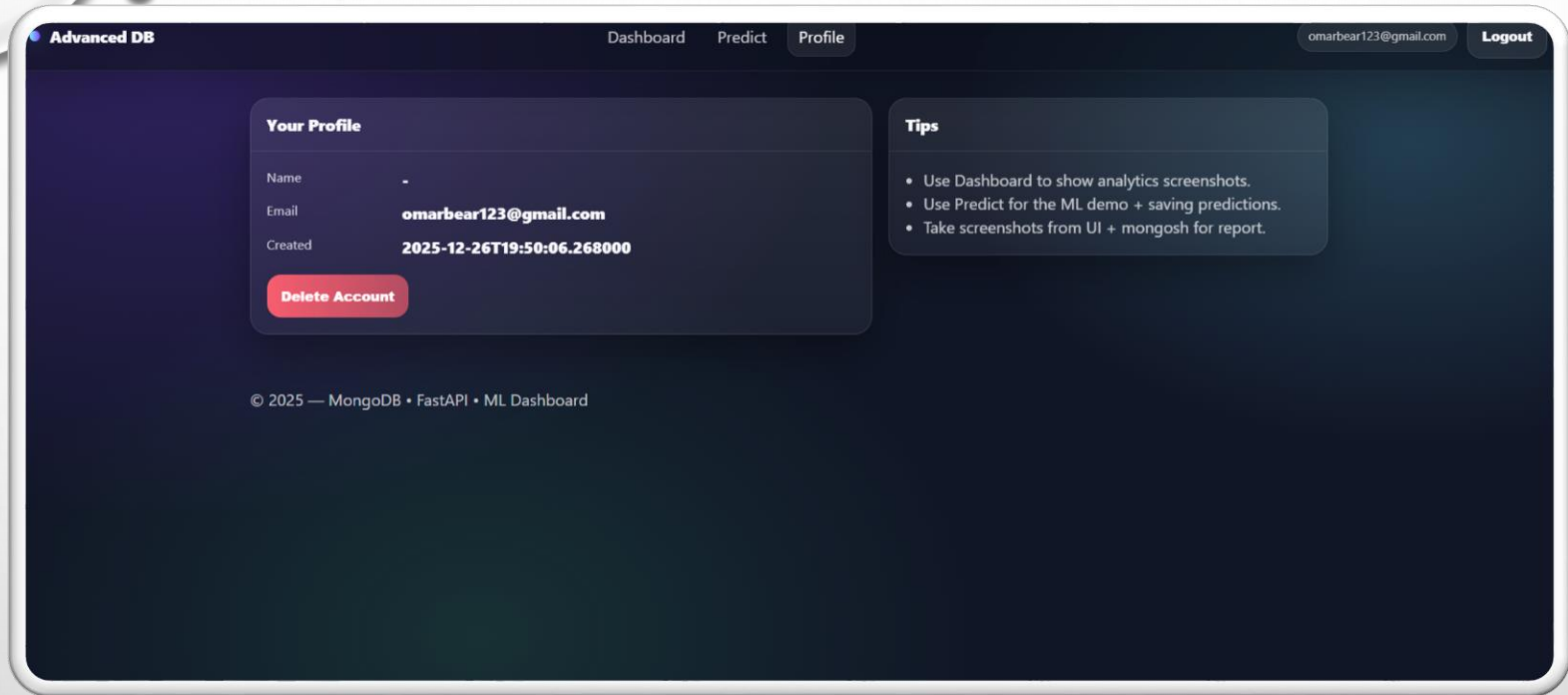
SAVES PREDICTION RESULTS TO THE DATABASE



PROFILE PAGE

DISPLAYS LOGGED-IN USER INFORMATION

ALLOWS LOGOUT OR ACCOUNT DELETION WITH
TRANSACTION SAFETY



PROFILE PAGE

DISPLAYS LOGGED-IN USER INFORMATION

**ALLOWS LOGOUT OR ACCOUNT DELETION WITH
TRANSACTION SAFETY**

DATA PREPROCESSING

- CLEANED RAW EBAY LAPTOP DATASET BY REMOVING DUPLICATES AND HANDLING MISSING VALUES
- CONVERTED CATEGORICAL FEATURES (CONDITION, COUNTRY, PROCESSOR) INTO CONSISTENT FORMATS
- NORMALIZED NUMERIC COLUMNS SUCH AS PRICE, RAM_GB, STORAGE_GB, AND SHIPPING_COST
- PREPARED FINAL DATASET FOR ANALYTICS, AGGREGATION, AND MACHINE LEARNING TRAINING

DATA PREPROCESSING

```
# Drop unnecessary columns
cols_to_drop = ['Promo_Condition_Mixed', 'Condition_Brand_Mixed', 'icon href 4',
                'su-styled-text 15', 'Refurbished_Label', 'Shipping', 'Watchers',
                'Feedback', 'Location', 'Listing_Type']
df_clean = df.drop(columns=[c for c in cols_to_drop if c in df.columns]).copy()

# Drop rows without target (Price)
initial_rows = df_clean.shape[0]
df_clean = df_clean.dropna(subset=['Price'])

# Impute Missing Values
for col in ['RAM_GB', 'Storage_GB', 'Screen_Size_Inch', 'Seller_Score', 'Seller_Reviews']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
df_clean['Processor'] = df_clean['Processor'].fillna('Other')
df_clean['Condition'] = df_clean['Condition'].replace('Unknown', df_clean['Condition'].mode()[0])

# Outlier Removal (IQR Rule)
Q1 = df_clean['Price'].quantile(0.25)
Q3 = df_clean['Price'].quantile(0.75)
IQR = Q3 - Q1
lower_bound, upper_bound = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR

df_final = df_clean[(df_clean['Price'] >= lower_bound) & (df_clean['Price'] <= upper_bound)].copy()

print(f"✓ Data Cleaning Report:")
print(f"  - Rows before cleaning: {initial_rows}")
print(f"  - Rows after cleaning & outlier removal: {df_final.shape[0]}")
print(f"  - Removed {initial_rows - df_final.shape[0]} rows (Missing Prices or Outliers).")
```

```
✓ Data Cleaning Report:
- Rows before cleaning: 7140
- Rows after cleaning & outlier removal: 6742
- Removed 398 rows (Missing Prices or Outliers).
```

DATA PREPROCESSING

```
# Apply Renaming
df = df.rename(columns=rename_map)

# Feature Extraction
print("⚙️ Extracting Features from Titles...")
df[['RAM_GB', 'Storage_GB', 'Processor', 'Screen_Size_Inch']] = df['Title'].apply(extract_specs)

print("⚙️ Standardizing Conditions...")
df['Condition'] = df.apply(extract_condition, axis=1)

# Numerical Conversions
print("⚙️ Cleaning Numerical Columns (Price, Shipping, Feedback)...")
df['Price'] = pd.to_numeric(df['Price'].astype(str).str.replace(r'[$,]', '', regex=True), errors='coerce')

# Shipping Helper
def clean_shipping(s):
    if pd.isna(s) or 'free' in str(s).lower(): return 0.0
    match = re.search(r'(\d+\.\d*)', str(s))
    return float(match.group(1)) if match else 0.0
df['Shipping_Cost'] = df['Shipping'].apply(clean_shipping)

# Feedback Helper
def clean_feedback(f):
    if pd.isna(f): return np.nan, np.nan
    score = float(re.search(r'(\d+(?:\.\d+)?)%', str(f)).group(1)) if re.search(r'%', str(f)) else np.nan #type: ignore
    count_match = re.search(r'\\([\\d\\.]+)([KM]?)\\', str(f))
    count = 0
    if count_match:
        num, suffix = float(count_match.group(1)), count_match.group(2)
        count = int(num * (1000 if suffix == 'K' else 1000000 if suffix == 'M' else 1))
    return score, count
df[['Seller_Score', 'Seller_Reviews']] = df['Feedback'].apply(lambda x: pd.Series(clean_feedback(x)))

# Categorical
df['Country'] = df['Location'].astype(str).str.replace('Located in ', '').str.strip()
df['Listing_Type_Cat'] = df['Listing_Type'].apply(lambda x: 'Auction' if 'bid' in str(x).lower() else 'Fixed Price')

print("✅ Data Processing Complete.")
```

- ⚙️ Extracting Features from Titles...
- ⚙️ Standardizing Conditions...
- ⚙️ Cleaning Numerical Columns (Price, Shipping, Feedback)...
- ✅ Data Processing Complete.

Machine Learning Model

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb

# --- Random Forest ---
rf_pipe = Pipeline([('preprocessor', preprocessor), ('model', RandomForestRegressor(random_state=42))])
rf_search = RandomizedSearchCV(rf_pipe, {'model__n_estimators': [100, 200], 'model__max_depth': [10, 20]},
                                n_iter=3, cv=3, scoring='neg_mean_squared_error', random_state=42)
rf_search.fit(X_train, y_train)
best_rf = rf_search.best_estimator_

# --- XGBoost ---
xgb_pipe = Pipeline([('preprocessor', preprocessor), ('model', xgb.XGBRegressor(random_state=42))])
xgb_search = RandomizedSearchCV(xgb_pipe, {'model__learning_rate': [0.01, 0.1], 'model__max_depth': [3, 6]},
                                n_iter=3, cv=3, scoring='neg_mean_squared_error', random_state=42)
xgb_search.fit(X_train, y_train)
best_xgb = xgb_search.best_estimator_

print("Models trained successfully.")
```

[11]

Python

... Models trained successfully.

Machine Learning Evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

for name, model in {"Random Forest": best_rf, "XGBoost": best_xgb}.items():
    preds = model.predict(X_test) # type: ignore
    print(f"[{name}]")
    print(f"RMSE: {np.sqrt(mean_squared_error(y_test, preds)):.2f}")
    print(f"MAE: {mean_absolute_error(y_test, preds):.2f}")
    print(f"R2: {r2_score(y_test, preds):.4f}\n")
```

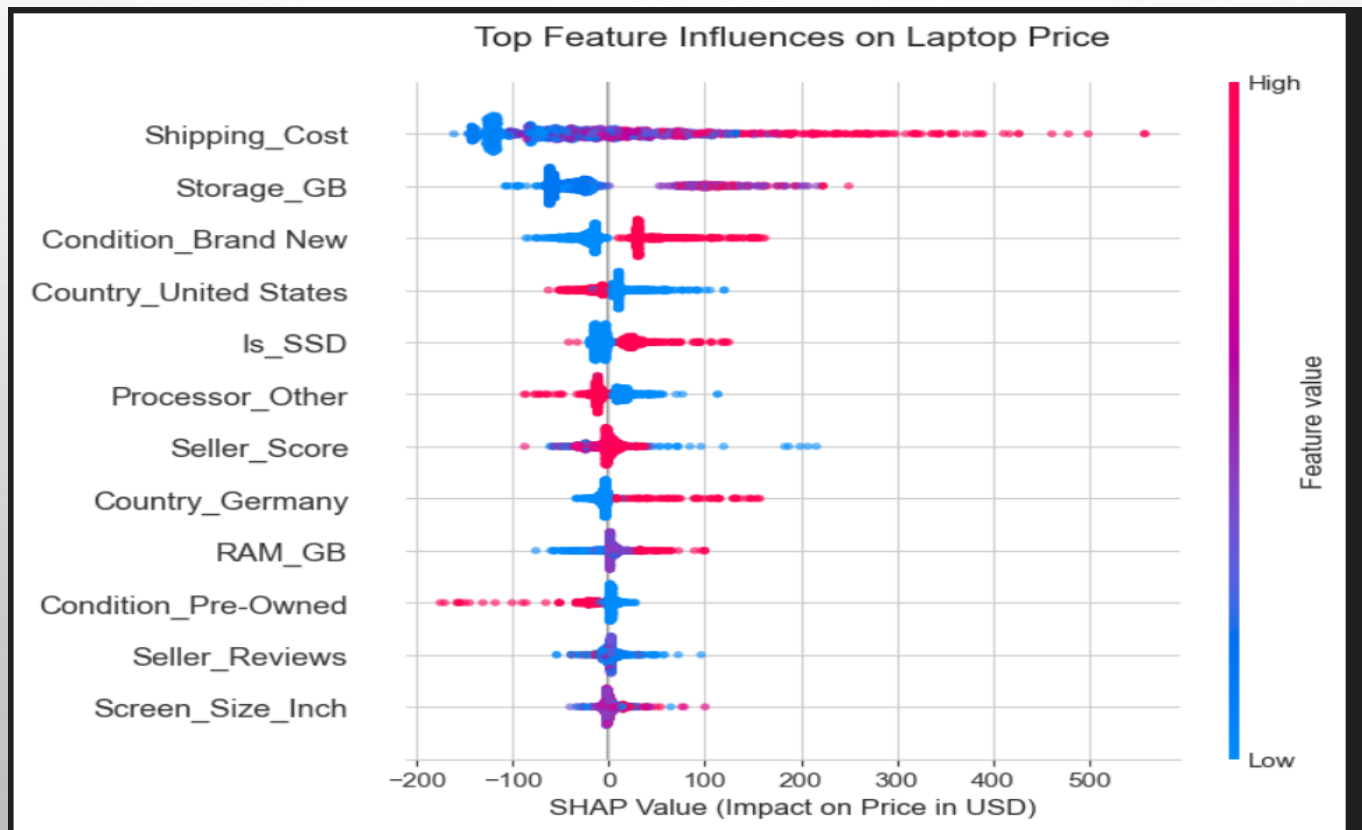
[12]

Python

```
... [Random Forest]
RMSE: 87.98
MAE: 48.94
R2: 0.8464
```

```
[XGBoost]
RMSE: 88.48
MAE: 55.41
R2: 0.8446
```


Result Explainable AI Shap



MODEL DEPLOYMENT

- Exported The Trained **Random Forest Model** And Loaded It In The Fastapi Backend At Application Startup
- Exposed A Secure **/Predict API Endpoint** That Receives Input Features And Returns Real-time Price Predictions
- Integrated The Deployed Model With The Frontend **Predict Page** For Live User Interaction
- Stored Prediction Inputs And Outputs In *Mongodb* To Enable Monitoring, Analytics, And Future Model Improvements

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, some in the top-left corner, some in the top-right, and a cluster in the bottom-right. In the center, there is a faint, circular logo or watermark that is not clearly legible.

CRUD Operations

CRUD Operations

Create:

Users can sign up, log in, and create new prediction records that are saved in MongoDB.

•Read:

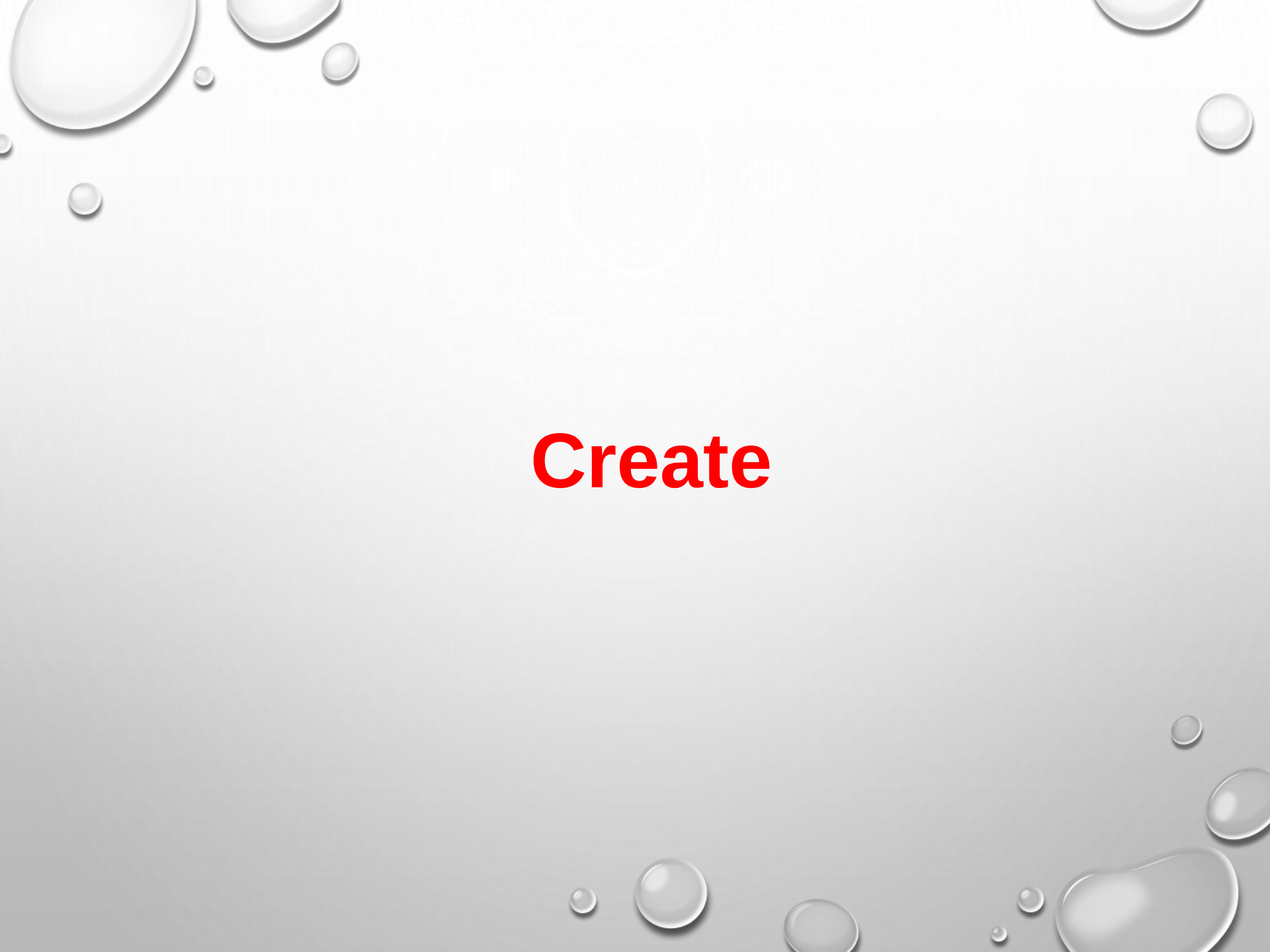
The system retrieves user profiles, prediction history, and analytics data from the database.

•Update:

User data (such as profile information) and system metadata (timestamps, usage stats) are updated when actions occur.

•Delete:

Users can delete their accounts, and all related predictions are removed atomically using MongoDB transactions.

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, primarily located in the top-left, top-right, and bottom-right corners. In the center of the image, there is a faint, circular, embossed-style logo. The logo appears to be a seal or crest, possibly representing an institution or organization, though its details are too light to discern clearly.

Create

```
{
  "name": "string",
  "email": "Drosamabedawey@example.com",
  "password": "123456"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/signup' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "string",
    "email": "Drosamabedawey@example.com",
    "password": "123456"
  }'
```

Request URL

http://127.0.0.1:8000/signup

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "message": "User created",
  "user_id": "6950541c9d5b08ef07d338db"
}
```



Download

Create

USER SIGNUP API – CREATING A NEW USER IN DATABASE

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.insertOne({
  .. name: "Test User",
  .. email: "test@example.com",
  .. password: "123456",
  .. created_at: new Date()
  .. })
..
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.insertOne({
  .. name: "Test User",
  .. email: "mosalah@example.com",
  .. password: "123456",
  .. created_at: new Date()
  .. })
..
{
  acknowledged: true,
  insertedId: ObjectId('695055c2a1f1b4003c1e2622')
}
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

Create

User Signup API Request via Swagger UI


```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({}, {email:1}).limit(10)
[
  {
    _id: ObjectId('694c47db5e6aaef6561e2621'),
    email: 'test@example.com'
  },
  {
    _id: ObjectId('694edfe6e87e80387d1e2621'),
    email: 'test123@example.com'
  },
  {
    _id: ObjectId('694ee65936bea039635641f6'),
    email: 'ahmed@example.com'
  },
  {
    _id: ObjectId('694ee6ee36bea039635641f7'),
    email: 'omarbear123@gmail.com'
  },
  {
    _id: ObjectId('694f1a2683d9f8c8dbd83274'),
    email: 'karim2004@gmail.com'
  },
  {
    _id: ObjectId('695053fc9d5b08ef07d338da'),
    email: 'Drosama@example.com'
  },
  {
    _id: ObjectId('6950541c9d5b08ef07d338db'),
    email: 'Drosamabedaway@example.com'
  }
]
```

Create

FETCHING EXISTING USERS FROM MONGODB (EMAIL PROJECTION)

```
{
  "email": "Drosamabedawey@example.com",
  "password": "123456"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "Drosamabedawey@example.com",
    "password": "123456"
  }'
```

Request URL

http://127.0.0.1:8000/login

Server response

Code Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2OTUwNTQxYzlkbnIwOGVmMDDkMzY4ZGIiLCJleHAiOjE3NjY5NTkxMzV9.6nsNqd6XdN_3Gm1UYVX-1317nE8JDSRWA03Q5F7twFY",
  "token_type": "bearer"
}
```

 Download

Create

USER LOGIN API – JWT TOKEN GENERATION

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.insertOne({  
..   user_id: ObjectId("694edfe6e87e80387d1e2621"),  
..   prediction: 250,  
..   model_version: "rf_v1",  
..   created_at: new Date()  
.. })  
  
acknowledged: true,  
insertedId: ObjectId('69505649a1f1b4003c1e2623')  
  
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project>
```

Create

CREATING A NEW PREDICTION RECORD IN MONGODB

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.insertOne({  
..   user_id: ObjectId("694edfe6e87e80387d1e2621"),  
..   prediction: 250,  
..   model_version: "rf_v1",  
..   created_at: new Date()  
.. })  
  
acknowledged: true,  
insertedId: ObjectId('69505649a1f1b4003c1e2623')  
  
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project>
```

Create

CREATING A NEW PREDICTION RECORD IN MONGODB

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, primarily located in the top-left, top-right, and bottom-right corners. In the center of the image, there is a faint, circular, embossed-style logo. The logo features a central emblem surrounded by text in a circular arrangement, though the details are too light to read clearly.

Read


```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({ email: "test@example.com" })
[
  {
    _id: ObjectId('694c47db5e6aaef6561e2621'),
    name: 'test',
    email: 'test@example.com',
    password: '123456',
    created_at: ISODate('2025-12-24T20:06:51.635Z'),
    updated_at: ISODate('2025-12-26T19:39:12.122Z')
  }
]
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project>
```

Read

READING USER DATA FROM MONGODB BY EMAIL


```

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.aggregate([
...   {
...     $lookup: {
...       from: "users",
...       localField: "user_id",
...       foreignField: "_id",
...       as: "user"
...     }
...   },
...   { $unwind: "$user" }
... ])
...
[
  {
    _id: ObjectId('694c47e35e6aaef6561e2622'),
    user_id: ObjectId('694c47db5e6aaef6561e2621'),
    input: { RAM_GB: 16 },
    prediction: 250,
    model_version: 'rf_v1',
    created_at: ISODate('2025-12-24T20:06:59.110Z'),
    user: {
      _id: ObjectId('694c47db5e6aaef6561e2621'),
      name: 'test',
      email: 'test@example.com',
    }
  }
]

```

Read

READING PREDICTION DATA WITH USER DETAILS USING MONGODB AGGREGATION

GET

/dashboard/stats

Dashboard Stats

Parameters

Name	Description
limit	
integer	5000
(query)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/dashboard/stats?limit=5000' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/dashboard/stats?limit=5000
```

Server response

Code	Details
200	Response body <pre>{ "columns": ["Title", "Price", "Seller_Name", "RAM_GB", "Storage_GB", "Processor", "Screen_Size_Inch", "Condition", "Shipping_Cost", "Seller_Score", "Seller_Reviews", "Country", "Listing_Type_cat", "created_at", "updated_at"] }</pre>

Read

DASHBOARD STATISTICS API – READING ANALYTICAL DATA (GET /DASHBOARD/STATS)

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, primarily located in the top-left, top-right, and bottom-right corners. In the center of the image, there is a faint, circular, embossed-style logo that appears to be a university crest or seal.

Update

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.updateOne(
...   { email: "test@example.com" },
...   { $set: { name: "Updated Name", updated_at: new Date() } }
... )
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project>
```

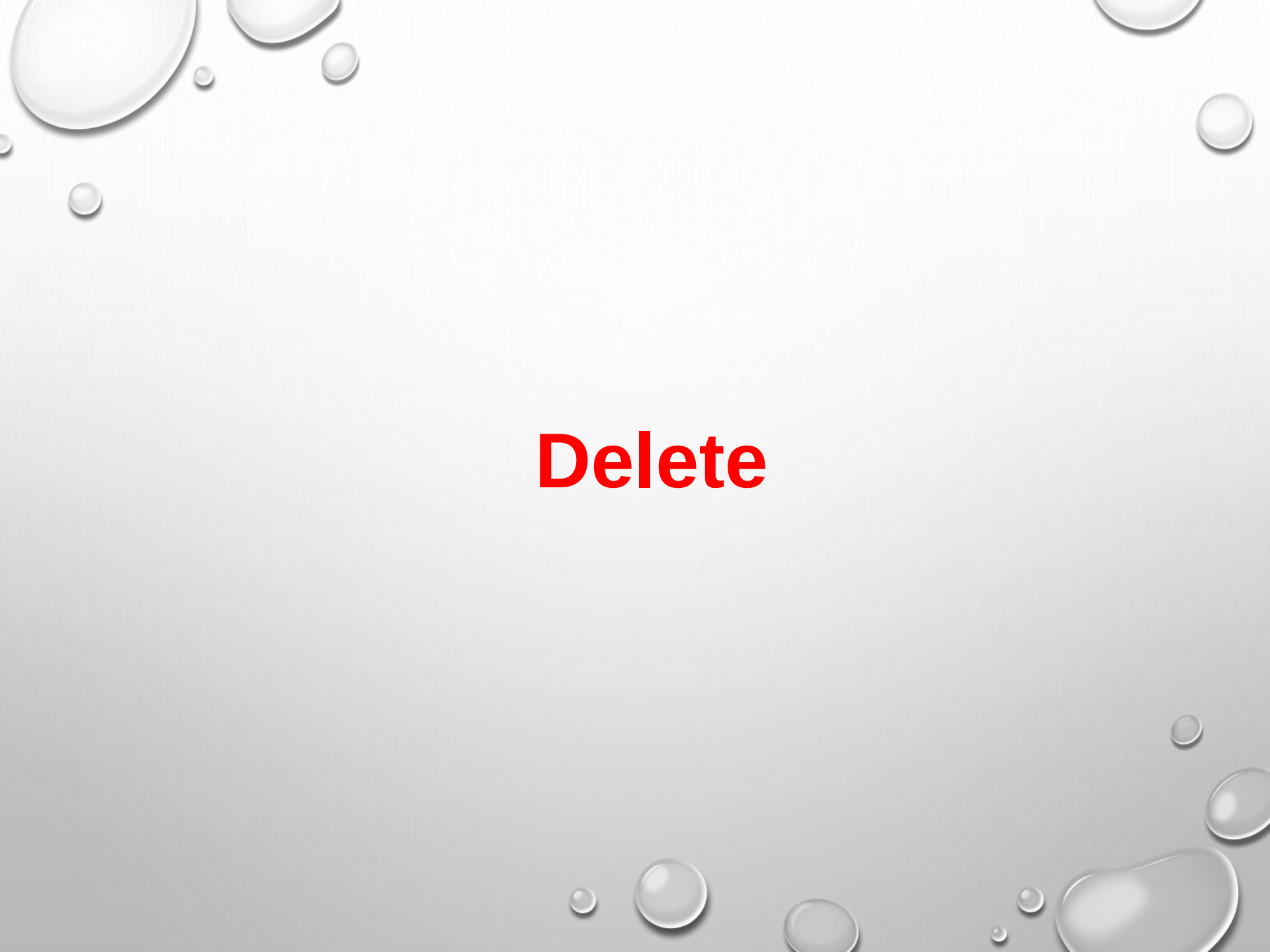
Update

UPDATING USER INFORMATION IN MONGODB USING UPDATEONE

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.updateOne(
...   { _id: ObjectId("6950541c9d5b08ef07d338db") },
...   { $inc: { usage_count: 1 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Update

**INCREMENTING USER USAGE COUNT USING MONGODB
\$INC OPERATOR**

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, primarily located in the top-left, top-right, and bottom-right corners. In the center of the image, there is a faint, circular, embossed-style logo. The logo appears to be a seal or crest, but its details are too light to discern clearly.

Delete

Parameters

[Cancel](#)

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \  
  'http://127.0.0.1:8000/delete-account' \  
  -H 'accept: application/json' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2OTUwNTQxYzlkNWl0OGVhMDdkMzY4ZGIiLCJleHAiOjE3NjY5NTk0dMzV9.6nsNqd6XdN_3Gm1UYX-1317nE8JDSRWA03QSF7twFY'
```

Request URL

<http://127.0.0.1:8000/delete-account>

Server response

Code Details

200

Response body

```
{  
  "message": "Account deleted successfully",  
  "user_id": "6950541c9d5b08ef07d338db"  
}
```



Download

Response headers

```
access-control-allow-credentials: true  
content-length: 79  
content-type: application/json  
date: Sat, 27 Dec 2025 22:08:48 GMT  
server: uvicorn
```

Responses

Code Description

[Links](#)

Delete

DELETING USER ACCOUNT USING SECURE API ENDPOINT (DELETE /DELETE-ACCOUNT)

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.find({ user_id: ObjectId("6950541c9d5b08ef07d338db") })
```

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

Delete

**VERIFYING PREDICTION RECORDS REMOVAL AFTER USER
DELETION**

The slide features a light gray gradient background. In the top-left and bottom-right corners, there are clusters of realistic water droplets of various sizes, rendered with soft shadows and highlights to give them a three-dimensional appearance. Faint, circular, embossed patterns are visible in the background, particularly in the upper half of the slide.

Indexing & Aggregation Pipelines

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.createIndex({ email: 1 }, { unique: true })  
email_1
```

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

INDEXING

```

...   user_id: db.users.findOne({ email: "test@example.com" })._id
... })
... .sort({ created_at: -1 })
... .limit(20)
... .explain("executionStats")
...
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'advanced_db_project.predictions',
    parsedQuery: { user_id: { '$eq': ObjectId('694c47db5e6aaef6561e2621') } },
    indexFilterSet: false,
    queryHash: '3A353FB0',
    planCacheShapeHash: '3A353FB0',
    planCacheKey: '877CF48E',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 20,
    }
  }
}

```

INDEXING

predictions.user_id index → fast user prediction history

```

as atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({ email: "test@example.com" }).explain("executionStats")

explainVersion: '1',
queryPlanner: {
  namespace: 'advanced_db_project.users',
  parsedQuery: { email: { '$eq': 'test@example.com' } },
  indexFilterSet: false,
  queryHash: 'ED28E3D2',
  planCacheShapeHash: 'ED28E3D2',
  planCacheKey: '1E950542',
  optimizationTimeMillis: 0,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  prunedSimilarIndexes: false,
  winningPlan: {
    isCached: false,
    stage: 'EXPRESS_IXSCAN',
    keyPattern: '{ email: 1 }',
    indexName: 'email_1'
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  returns: 1
}

```

INDEXING

MongoDB Query Execution Plan Using Email Index (EXPLAIN Output)

```

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.dataset.aggregate([
...   {
...     $group: {
...       _id: null,
...       count: { $sum: 1 },
...       avgPrice: { $avg: "$Price" },
...       minPrice: { $min: "$Price" },
...       maxPrice: { $max: "$Price" }
...     }
...   }
... ])
...
... [
...   {
...     _id: null,
...     count: 6742,
...     avgPrice: 269.2871566300801,
...     minPrice: 3.04,
...     maxPrice: 957.88
...   }
... ]
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |

```

Aggregation

MongoDB Aggregation Result: Dataset Price Statistics (Count, Average, Min, Max)

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.aggregate([
...   {
...     $lookup: {
...       from: "users",
...       localField: "user_id",
...       foreignField: "_id",
...       as: "user"
...     }
...   },
...   { $unwind: "$user" },
...   {
...     $project: {
...       _id: 0,
...       user_email: "$user.email",
...       prediction: 1,
...       model_version: 1,
...       created_at: 1
...     }
...   },
...   { $sort: { created_at: -1 } },
...   { $limit: 10 }
... ])
...
[
  {
    prediction: 250,
    model_version: 'rf_v1',
    created_at: ISODate('2025-12-24T20:06:59.110Z'),
    user_email: 'test@example.com'
  }
]
```

Aggregation

MongoDB Aggregation with \$lookup: Joining Predictions and Users (Latest Results)

```
as atlas-v19wdp-shard-0 [primary] advanced_db_project> db.dataset.aggregate([
  {
    $bucket: {
      groupBy: "$Price",
      boundaries: [0, 200, 400, 600, 800, 1000, 1500, 2000, 5000],
      default: "Other",
      output: { count: { $sum: 1 } }
    }
  }
])

{ "_id": 0, "count": 3369 },
{ "_id": 200, "count": 1786 },
{ "_id": 400, "count": 870 },
{ "_id": 600, "count": 457 },
{ "_id": 800, "count": 260 }

as atlas-v19wdp-shard-0 [primary] advanced_db_project> |
```

Aggregation

MongoDB \$bucket Aggregation: Price Distribution Histogram


```
as atlas-vl9wdp-shard-0 [primary] advanced_db_project> db.dataset.aggregate([
  { $group: { _id: "$Condition", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 10 }
])

_id: 'Brand New', count: 2983 },
_id: 'Pre-Owned', count: 1413 },
_id: 'Very Good - Refurbished', count: 615 },
_id: 'Good - Refurbished', count: 579 },
_id: 'Open Box', count: 382 },
_id: 'Excellent - Refurbished', count: 327 },
_id: 'Parts Only', count: 193 },
_id: 'Certified - Refurbished', count: 165 },
_id: 'Shop With Confidence, Buy From Newegg', count: 20 },
_id: 'New (Other)', count: 20 }

as atlas-vl9wdp-shard-0 [primary] advanced_db_project> |
```

Aggregation

MongoDB Aggregation: Top 10 Product Conditions by Count


```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.dataset.aggregate([
...   { $group: { _id: "$Country", count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 10 }
... ])
...
[
  { _id: 'United States', count: 2797 },
  { _id: 'France', count: 1694 },
  { _id: 'United Kingdom', count: 775 },
  { _id: 'Germany', count: 504 },
  { _id: 'Italy', count: 501 },
  { _id: 'Spain', count: 207 },
  { _id: 'Canada', count: 49 },
  { _id: 'China', count: 48 },
  { _id: 'Australia', count: 48 },
  { _id: 'Japan', count: 38 }
]
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

Aggregation

MongoDB Aggregation: Top 10 Countries by Dataset Count

```

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.dataset.aggregate([
...   {
...     $group: {
...       _id: "$Condition",
...       avgPrice: { $avg: "$Price" },
...       count: { $sum: 1 }
...     }
...   },
...   { $sort: { avgPrice: -1 } }
... ])
...
[
  {
    _id: 'Twin Batteries New Charger Stylus Military Grade #G',
    avgPrice: 652.22,
    count: 2
  },
  {
    _id: 'Brand New - Full Warranty',
    avgPrice: 568.3233333333334,
    count: 6
  }
]

```

Aggregation

MongoDB Aggregation: Average Price by Product Condition (Sorted Descending)

The background is a light gray gradient. It is decorated with several realistic water droplets of various sizes, some in the top-left, top-right, and bottom-right corners. In the center, there is a faint, circular seal or logo, which appears to be the official seal of the University of Cambridge, featuring a crest and Latin text.

Transactions, Replication

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> rs.status()
{
  set: 'atlas-yl9wdp-shard-0',
  date: ISODate('2025-12-26T18:44:29.800Z'),
  myState: 1,
  term: Long('77'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    lastCommittedWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    appliedOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    durableOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    writtenOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    lastAppliedWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    lastDurableWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    lastWrittenWallTime: ISODate('2025-12-26T18:44:27.383Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1766774661, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'stepUpRequestSkipDryRun',
    lastElectionDate: ISODate('2025-12-17T17:08:39.114Z'),
    electionTerm: Long('77'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1765991319, i: 1 }), t: Long('76') },
  }
}
```

Transactions, Replication

MongoDB Replica Set Status: Replication Health & Transaction Commit State


```

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> rs.status()
{
  set: 'atlas-yl9wdp-shard-0',
  date: ISODate('2025-12-26T18:44:29.800Z'),
  myState: 1,
  term: Long('77'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    lastCommittedWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    appliedOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    durableOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    writtenOpTime: { ts: Timestamp({ t: 1766774667, i: 1 }), t: Long('77') },
    lastAppliedWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    lastDurableWallTime: ISODate('2025-12-26T18:44:27.383Z'),
    lastWrittenWallTime: ISODate('2025-12-26T18:44:27.383Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1766774661, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'stepUpRequestSkipDryRun',
    lastElectionDate: ISODate('2025-12-17T17:08:39.114Z'),
    electionTerm: Long('77'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1765991319, i: 1 }), t: Long('76') },
  }
}

```

Transactions

MongoDB Replica Set Status: Replication Health & Transaction Commit State

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({ email: "test@example.com" }, { email: 1 })
[
  {
    _id: ObjectId('694c47db5e6aaef6561e2621'),
    email: 'test@example.com'
  }
]

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.countDocuments({ user_id: ObjectId("694c47db5e6aaef6561e2621") })
1

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.countDocuments({ _id: ObjectId("694c47db5e6aaef6561e2621") })
1

Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

Activate Windows
Go to Settings to activate Windows

Transactions

MongoDB Verification Before Deletion: User Existence & Related Predictions Count

http://127.0.0.1:8080/delete-account

Server response

Code	Details	Links						
200	Response body <pre>{ "message": "Account deleted successfully", "user_id": "694ee95136bea039635641f8" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 79 content-type: application/json date: Fri, 26 Dec 2025 20:00:36 GMT server: uvicorn</pre>	Download						
Responses <table><thead><tr><th>Code</th><th>Description</th><th>Links</th></tr></thead><tbody><tr><td>200</td><td>Successful Response</td><td>No links</td></tr></tbody></table>			Code	Description	Links	200	Successful Response	No links
Code	Description	Links						
200	Successful Response	No links						

Media type:

Controls Accept header.

Example Value | Schema

```
"string"
```

Analytics

Transactions

API Response: Successful User Account Deletion (HTTP 200)

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.countDocuments({ user_id: ObjectId("694ee59036bea039635641f5") })
```

0

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.countDocuments({ _id: ObjectId("694ee59036bea039635641f5") })
```

0

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

Activate Windows

Go to Settings to activate Windows

Transactions

MongoDB Verification After Deletion: User and Related Predictions Successfully Removed

The background is a light gray gradient. In the top-left and bottom-right corners, there are several realistic water droplets of various sizes, rendered with highlights and shadows. In the center of the image, there is a faint, circular, embossed-style logo. The logo appears to be a seal or crest, possibly of a university or institution, but the details are too light to discern clearly. It features a central emblem surrounded by text in a circular border.

ROLLBACK

```
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({}, {email:1}).limit(5)
[
  {
    _id: ObjectId('694c47db5e6aaef6561e2621'),
    email: 'test@example.com'
  },
  {
    _id: ObjectId('694edfe6e87e80387d1e2621'),
    email: 'test123@example.com'
  },
  {
    _id: ObjectId('694ee65936bea039635641f6'),
    email: 'ahmed@example.com'
  },
  {
    _id: ObjectId('694ee6ee36bea039635641f7'),
    email: 'omarbear123@gmail.com'
  },
  {
    _id: ObjectId('694eea4d36bea039635641f9'),
    email: 'engmazen@example.com'
  }
]
Atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

ROLLBACK

BEFORE ROLLBACK – USER AND PREDICTIONS NOT FOUND

http://127.0.0.1:8080/delete-account

Server response

Code	Details						
200	Response body <pre>{ "message": "Account deleted successfully", "user_id": "694ee95136bea039635641f8" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 79 content-type: application/json date: Fri, 26 Dec 2025 20:00:36 GMT server: uvicorn</pre> Responses <table><thead><tr><th>Code</th><th>Description</th><th>Links</th></tr></thead><tbody><tr><td>200</td><td>Successful Response</td><td>No links</td></tr></tbody></table> Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	Code	Description	Links	200	Successful Response	No links
Code	Description	Links					
200	Successful Response	No links					

Analytics

ROLLBACK

**ACCOUNT DELETION API EXECUTION – ROLLBACK
TRIGGERED SUCCESSFULLY**

```
atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.find({}, {email:1}).limit(5)
{
  "_id": ObjectId('694c47db5e6aaef6561e2621'),
  "email": "test@example.com"
},
{
  "_id": ObjectId('694edfe6e87e80387d1e2621'),
  "email": "test123@example.com"
},
{
  "_id": ObjectId('694ee65936bea039635641f6'),
  "email": "ahmed@example.com"
},
{
  "_id": ObjectId('694ee6ee36bea039635641f7'),
  "email": "omarbear123@gmail.com"
},
{
  "_id": ObjectId('694eea4d36bea039635641f9'),
  "email": "engmazen@example.com"
}

atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.predictions.countDocuments({ user_id: ObjectId("694eea4d36bea039635641f9") })
1

atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> db.users.countDocuments({ _id: ObjectId("694eea4d36bea039635641f9") })
1

atlas atlas-yl9wdp-shard-0 [primary] advanced_db_project> |
```

ROLLBACK

AFTER ROLLBACK – USER AND PREDICTIONS RESTORED



CONCLUSION

CONCLUSION

- Built an end-to-end **full-stack application** integrating MongoDB Atlas, FastAPI, React, and Machine Learning.
- Applied **advanced database concepts** including aggregation pipelines, indexing, replication, and transactions.
- Delivered a **production-ready system** with analytics dashboards, ML predictions, and secure user workflows

DEMO

Advanced DB

AI

Advanced DB Project
MongoDB Atlas • FastAPI • ML

Create an account

Sign up to access analytics, predictions, and your profile.

Name

Kareem

Email

user@example.com

Password

Sign Up

Already have an account? [Login](#)

Activate Windows

Go to Settings to activate Windows.



ANY QUESTIONS?



THANK YOU!